

Rapport de Stage

Prépa des INP - Cycle Préparatoire Polytechnique

Étude des communications pour un réseau de robots mobiles

Stagiaire :

Amjad Fanid Kouanda

Encadrant :

Théo DOCQUIER

Laboratoire LORIA - Équipe SIMBIOT

Encadrant LORIA :

Campus scientifique
BP 239 54506 Vandœuvre-lès-Nancy Cedex
Tél : +33 3 83 59 20 00

Benjamin PHan

Période de Stage : 4 mai - 12 juin 2026
Année universitaire 2025-2026

Remerciements

Je souhaite en premier lieu remercier **Théo Docquier** pour ses explications et les axes de développement qu'il m'a suggérés, ainsi que **Benjamin Phan**, doctorant au LORIA, pour sa disponibilité, ses explications et la confiance qu'il m'a accordée afin de mener à bien ce projet. Mes remerciements vont aussi à **Matthieu**, doctorant au LORIA, pour sa pédagogie et son aide quant à la manipulation des équipements informatiques proposés. Enfin, je tiens à remercier sincèrement l'équipe **SYMBIOT** de m'avoir accueillis et donné accès aux ressources matérielles et logicielles nécessaires à ce projet.

Table des matières

1	Cadre et environnement du stage	4
1.1	Contexte du stage	4
1.2	Le LORIA	4
1.3	L'ingénieur de recherche	5
2	Introduction	6
2.1	Contexte du projet	6
2.2	Problématique et Objectif	6
3	Les protocoles de routage	7
3.1	Type de réseau utilisé	7
3.2	AODV	9
3.3	OLSR	10
3.4	BATMAN	12
3.5	Comparaison des différents protocoles	15
4	Campagne de tests	25
4.1	Environnement	25
4.2	Protocoles expérimentaux	26
5	Présentation des résultats	26
6	Conclusion	30
7	Bibliographie	31

1 Cadre et environnement du stage

1.1 Contexte du stage

Six semaines au sein d'un laboratoire de recherche : c'est l'expérience qu'a rendue possible mon stage de deuxième année à la Prépa des INP de Nancy. Accueilli par le LORIA, j'ai travaillé durant ce stage au sein de l'équipe SIMBIOT sur l'étude des communications au sein d'un réseau de robots mobiles. Ce travail s'articule en deux grandes phases : une première consacrée à la recherche bibliographique et à l'analyse théorique, suivie d'une mise en pratique. L'ensemble de cette démarche vise à répondre à la problématique suivante : **Dans quelle mesure l'adoption des protocoles de routage AODV, OLSR ou BATMAN serait-elle pertinente pour garantir la communication d'une flotte de robots mobiles dans un environnement souterrain ?**

1.2 Le LORIA

Mon stage se déroule donc au LORIA, un laboratoire lorrain de recherche en informatique et ses applications. C'est une unité mixte du Centre National de la Recherche Scientifique (CNRS : organisme de recherche public) et de l'Université de Lorraine, en partenariat avec l'INRIA et CentraleSupélec Metz.

Fondé en 1997, les travaux scientifiques au sein du LORIA sont menés par **27 équipes** structurées en **5 départements**, dont 14 sont communes à l'INRIA, présentant un total de près de 500 personnes.

Au sein de ce laboratoire on retrouve 5 axes d'étude :

- D1 - Algorithmique, calcul, image et géométrie
- D2 - Méthodes formelles
- D3 - Réseaux, systèmes et services
- D4 - Traitement automatique des langues et des connaissances
- D5 - Systèmes complexes, intelligence artificielle et robotique

Mon stage s'effectue au sein de l'axe D3, celui-ci s'intéresse à la façon dont plusieurs ordinateurs ou robots peuvent travailler ensemble, communiquer et partager des données efficacement, même à grande échelle. Les autres axes ne seront pas détaillés, mais sont présentés sur le site internet du LORIA.

Avec **12 lauréats ERC** (chercheurs ayant obtenu un financement du Conseil Européen de la Recherche, les montants pouvant aller de centaines de milliers jusqu'à 2 millions d'euros), **500 publications par an** dans des articles ou revues, ainsi que **19 millions d'euros de contrats**, le LORIA est l'un des plus grands laboratoires de la région Grand Est et, avec plus de **40 nationalités** en son sein, il se distingue par une communauté scientifique ouverte sur le monde.

1.3 L'ingénieur de recherche

Au LORIA, on distingue plusieurs profils.

L'enseignant-chercheur qui lui partage son temps entre l'enseignement dans le supérieur et la recherche. Titulaire d'une thèse, il débute sa carrière en tant que maître de conférences avant de pouvoir évoluer vers le grade de professeur. Il est également habilité à encadrer des doctorants.

L'ingénieur de recherche, quant à lui, représente le grade le plus élevé de la filière ingénieur. Son rôle est d'apporter un soutien scientifique et technique aux enseignants-chercheurs, en consacrant la moitié de son temps à la recherche. Il peut aussi diriger des thèses, encadrer une équipe ou développer des partenariats avec l'industrie.

Enfin, **les doctorants, masters et stagiaires** sont encadrés par un enseignant-chercheur de l'Université de Lorraine."

2 Introduction

2.1 Contexte du projet

L'exploration et la supervision de zones d'enfouissement nucléaire est aujourd'hui assurée par des opérateurs humains, une pratique amenée à évoluer en raison de l'augmentation des niveaux de radioactivité dans ces zones. C'est dans ce contexte qu'un sujet de thèse a émergé entre Benjamin Phan, doctorant au LORIA et l'Andra, l'Agence nationale pour la gestion des déchets radioactifs. Ce sujet vise à confier ces missions à une flotte de robots mobiles (drones, robots quadrupèdes, etc.), devant interagir entre eux afin de mener à bien une mission commune, comme par exemple la supervision du site.

2.2 Problématique et Objectif

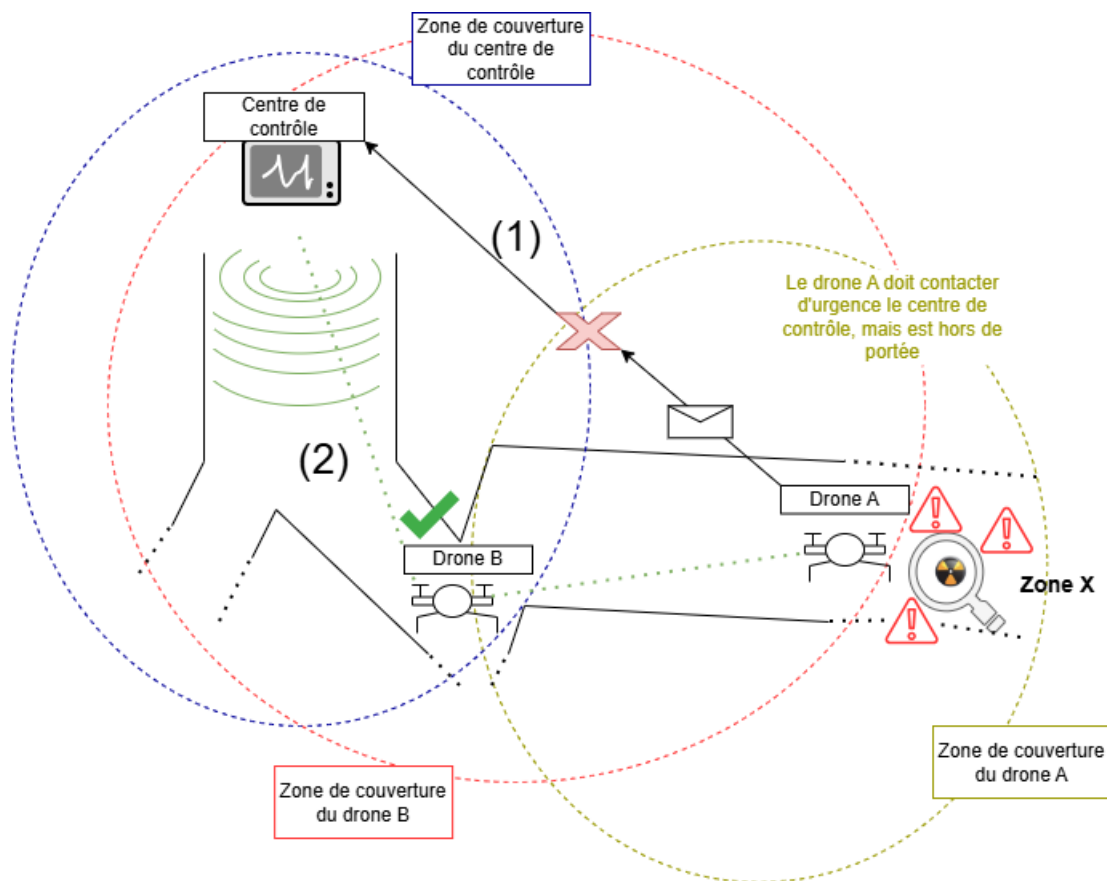


Figure N°1 : illustration de la problématique du sujet.

Dans l'image ci-dessus, supposons que le **drone d'exploration « A »** doive envoyer de toute urgence un message au centre de contrôle, ex : le seuil critique de radioactivité a été atteint dans la zone X. Pour ce faire, le drone A **contacte directement** le centre de contrôle (1), cependant

le message **échoue** car celui-ci est **hors de portée**. Maintenant, on ajoute un **drone B** qui est à portée du centre de contrôle et du drone A ; le message passe ainsi à travers le drone B, qui agit comme un **relais**, et arrive au centre de contrôle. C'est ce que l'on nomme le **routage**, on cherche un chemin dans un réseau pour acheminer une information.

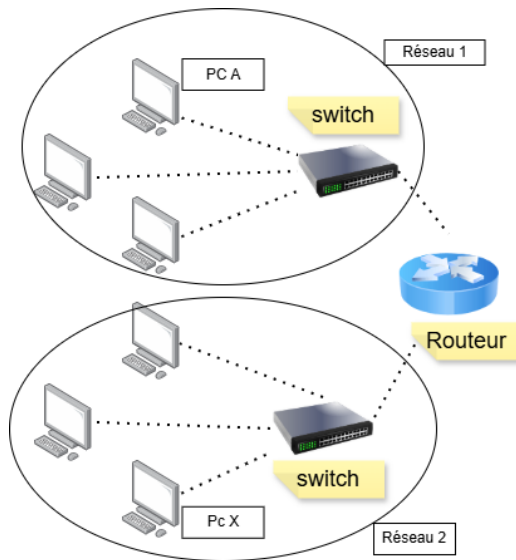
À ce jour, Benjamin Phan effectue des simulations en local, mais le projet final implique que ces robots mobiles puissent communiquer aisément et retransmettre des données selon certains critères.

L'objectif de ce stage concerne donc l'étude des différentes possibilités de communication au sein d'un système multi-agent mobile. Pour cela, différents protocoles de routage seront envisagés, tels qu'AODV, OLSR et BATMAN. Une première phase sera consacrée à l'étude et à la prise en main de ces différents protocoles ; une deuxième phase concernera une batterie de tests à l'aide d'une architecture simple, composée de microcontrôleurs, visant à reproduire un environnement cible ; et enfin la troisième étape sera l'évaluation des performances de ces méthodes ainsi que de la pertinence de leur adoption.

3 Les protocoles de routage

3.1 Type de réseau utilisé

L'environnement dans lequel évoluent les robots rend la gestion à distance difficile et l'installation d'équipements physiques fixes tels que des routeurs particulièrement fastidieuse. Dans ce contexte, le projet impose de recourir à un réseau ad hoc, c'est un type de réseau décentralisé permettant aux appareils de se connecter directement entre eux, sans infrastructure centrale. Dans ce réseau, chaque robot constitue ce que l'on appelle un nœud mobile, c'est-à-dire un équipement capable de se déplacer physiquement tout en maintenant sa connexion au réseau. À la différence d'un nœud fixe, tel qu'un serveur en datacenter, sa position au sein du réseau est donc amenée à évoluer continuellement.



Le réseau classique (infrastructure centralisée)

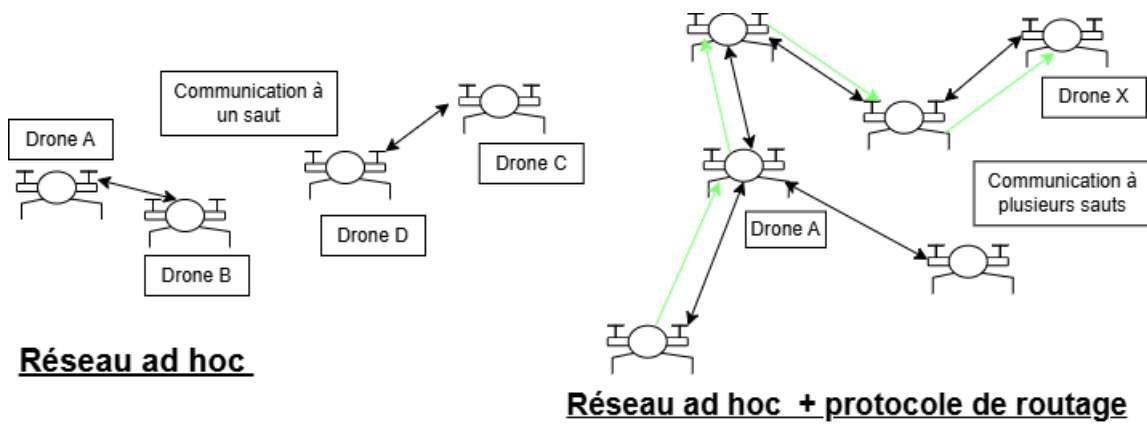
la communication repose sur un équipement central : le routeur.

PC A ne peut pas communiquer directement avec le PC X. Le message transite obligatoirement par le routeur, qui détermine le meilleur chemin pour acheminer l'information d'un réseau à un autre

Figure N°2 : Exemple d'infrastructure d'un réseau centralisé

switch : permet de connecter plusieurs appareils au sein d'un même réseau et dirige les données uniquement vers le destinataire concerné

Le réseau sans fil ad hoc a été initialement conçu pour des communications à un seul saut entre nœuds. C'est dans ce contexte qu'il devient nécessaire d'ajouter, par-dessus cette architecture réseau, un protocole de routage. Ce dernier se charge de maintenir à jour l'état du réseau et de déterminer le chemin optimal pour acheminer l'information jusqu'au nœud destinataire. Cet ajout permet ainsi de réaliser des communications multi-sauts.



Dans le cadre de notre **objectif**, qui est de permettre à des robots mobiles de communiquer efficacement en milieu contraint, il est nécessaire d'évaluer différents protocoles de routage.

3.2 AODV

Caractéristiques

AODV est un protocole de routage dit **réactif** : contrairement à une approche qui maintiendrait en permanence une cartographie du réseau, il ne cherche un chemin vers une destination que lorsqu'un nœud en a effectivement besoin. En l'absence de trafic, aucune ressource n'est consommée pour entretenir des routes qui ne servent pas, ce qui constitue un avantage non négligeable sachant que les robots disposent d'une autonomie limitée

Lorsqu'un nœud souhaite communiquer avec une destination pour laquelle il ne dispose d'aucune route, il déclenche une phase de découverte en diffusant un message RREQ (Route Request) à l'ensemble de ses voisins, qui le retransmettent à leur tour jusqu'à atteindre la destination ou un nœud intermédiaire disposant d'une route récente. Ce mécanisme, dit d'inondation, garantit de trouver un chemin mais introduit une latence initiale et peut saturer le réseau lorsque le nombre de messages devient important. Une fois la route trouvée, le nœud destination répond via un message RREP (Route Reply) qui remonte le chemin inverse. Si en cours de route un lien se rompt, un message RERR (Route Error) est émis pour notifier les nœuds concernés.

AODV ne retient qu'un seul chemin par destination : le meilleur, déterminé selon deux critères, le nombre de sauts d'abord, puis le numéro de séquence pour garantir de ne jamais emprunter une route devenue obsolète. Il ne dispose donc d'aucune route de secours en cas

de rupture, ce qui l'oblige à relancer une découverte complète. Pour éviter les conflits entre routes, chaque nœud incrémente son propre numéro de séquence avant d'initier une découverte ou d'émettre une réponse ; en cas de doute, la route portant le numéro le plus élevé est systématiquement conservée.

Dans notre contexte, ce comportement peut s'avérer problématique : chaque rupture de lien, fréquente dans un environnement souterrain où les robots sont en mouvement, déclenche une nouvelle inondation, introduisant des délais qui pourraient être critiques lors de la transmission d'une alerte urgente.

Dimensionnement

Le protocole AODV est conçu pour fonctionner dans des réseaux mobiles ad hoc de taille variable, pouvant aller de quelques dizaines à plusieurs milliers de nœuds mobiles.

3.3 OLSR

Caractéristiques

En comparaison avec AODV , OLSR est **un protocole proactif** , c'est à dire qu'il échange régulièrement des informations de topologie avec les autres nœuds du réseau . De ce fait la topologie du réseau est connu à chaque instant.

OLSR utilise **un routage saut par saut** : chaque nœud prend sa propre décision de transmission en se basant uniquement sur ses informations locales, sans que le paquet n'ait à connaître le chemin complet à l'avance.

La caractéristique la plus distinctive d'OLSR est l'utilisation des **relais multipoints (MPR)**.

L'idée des relais multipoints est de minimiser la surcharge liée à la diffusion de messages dans le réseau en réduisant les retransmissions redondantes au sein d'une même zone.

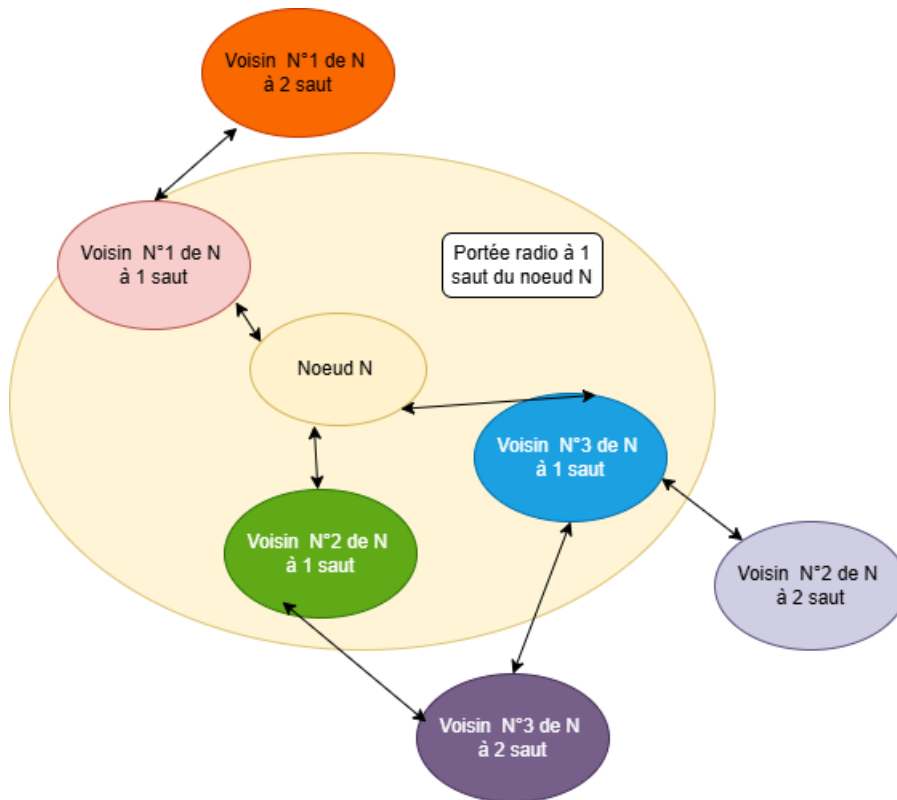


Figure N°3 : Illustration du mécanisme de sélection des MPR

Chaque nœud du réseau sélectionne un ensemble de nœuds dans son voisinage à 1 saut susceptibles de retransmettre ses messages. Cet ensemble est appelé l'ensemble MPR du nœud.

Les voisins d'un nœud N qui ne font pas partie de son ensemble MPR reçoivent et traitent les messages diffusés, mais ne les retransmettent pas. Chaque nœud sélectionne son ensemble MPR parmi ses voisins à un saut, de façon à couvrir (en termes de portée radio) tous les nœuds stricts à 2 sauts. L'ensemble MPR de N, noté $MPR(N)$, est un sous-ensemble du voisinage à 1 saut de N tel que chaque nœud du voisinage strict à 2 sauts de N dispose d'un lien vers au moins un nœud de $MPR(N)$.

Dans le cas du schéma ci dessous $MPR(N)$ comprend : "Voisin N°1 à 1 saut", "Voisin N°3 à 1 saut" mais ne contient pas "voisin N°2 à 1 saut" car il ne dispose pas de lien vers voisin N°2 à 2 saut.

Ce qui rend OLSR particulièrement efficace dans les réseaux de grande taille et à forte densité. Plus l'ensemble MPR est petit, moins le protocole de routage génère de surcharge de trafic de contrôle.

Fonctionnement

Pour maintenir sa connaissance du réseau, OLSR s'appuie sur deux types de messages échangés périodiquement. Les messages HELLO permettent à chaque nœud de détecter ses voisins directs et d'évaluer l'état des liens qui les relient, assurant ainsi une vision à jour du voisinage immédiat. Les messages TC (Topology Control), quant à eux, sont diffusés à l'ensemble du réseau mais uniquement par les nœuds désignés comme MPR, permettant à chaque nœud de reconstituer une carte partielle mais suffisante de la topologie globale. À partir de ces informations, chaque nœud calcule sa table de routage, les chemins empruntés passant systématiquement par les MPR, ce qui assure une cohérence avec la topologie connue.

Cependant, dans un environnement souterrain où la topologie évolue continuellement, le décalage temporel entre l'état réel du réseau et les tables de routage stockées pourrait générer des boucles de routage, situation dans laquelle un paquet tourne indéfiniment entre plusieurs nœuds sans jamais atteindre sa destination.

3.4 BATMAN

Tout comme le protocole OLSR, le protocole de routage BATMAN est **proactif**. La stratégie de BATMAN consiste à identifier, pour chaque destination du réseau, un voisin à saut unique pouvant servir de passerelle optimale pour communiquer avec ce nœud.

Pour cela, le protocole se base sur un type de message nommé OGM (Originator Message). Chaque nœud BATMAN diffuse régulièrement un OGM, informant ainsi ses voisins à 1 saut de son existence (première étape). Ces voisins, recevant les messages d'origine, les rediffusent selon les règles de routage spécifiques de BATMAN (deuxième étape), puis le processus se répète pour les voisins à 3 sauts, 4 sauts, et ainsi de suite. Le réseau est donc inondé d'OGMs jusqu'à ce que chaque nœud les ait reçus au moins une fois, ou jusqu'à ce qu'ils soient perdus en raison d'interférences, de collisions ou de l'expiration de leur TTL (durée de vie).

Le nombre d'OGMs reçus d'un émetteur donné via chaque voisin de liaison permet d'estimer la qualité d'une route. Ainsi l'on peut déterminer le meilleur chemin vers un émetteur donné, en comptabilisant les messages reçus de cet émetteur et enregistre le voisin local ayant relayé le plus grand nombre de ces messages. Grâce à ces informations, BATMAN maintient

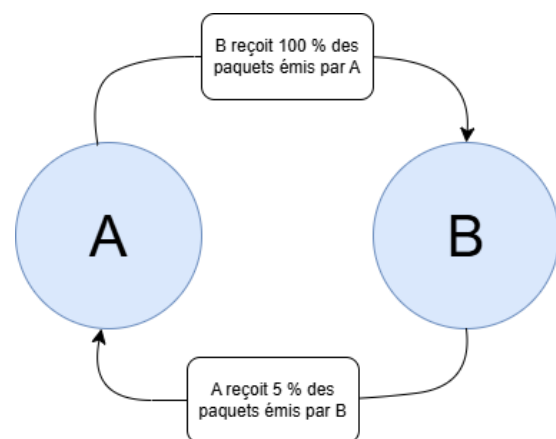
une table répertoriant le meilleur voisin local pour chaque émetteur du réseau.

Le protocole B.A.T.M.A.N. existe en plusieurs versions, dont les plus récentes sont les versions III, IV et V. Chacune d'elles apporte des améliorations ou résout des problèmes inhérents à la version précédente. Dans le cadre des tests effectués à la **section 5**, la version IV sera utilisée par souci de compatibilité avec le matériel disponible.

Cependant, si ces tests devaient être reconduits, un problème important de la version III mérite d'être soulevé : sa gestion des liens asymétriques. En effet, l'algorithme B.A.T.M.A.N. III présente des lacunes significatives à cet égard.

Le comportement de l'algorithme peut être ajusté via le délai pendant lequel B.A.T.M.A.N. accepte la retransmission de ses propres OGM par ses voisins. Un délai court rend l'algorithme très sélectif dans le choix des liens, au risque d'ignorer de nombreuses connexions fonctionnelles dans un seul sens, ne retenant ainsi que les liens parfaitement symétriques. À l'inverse, un délai plus long permet d'accepter un plus grand nombre de liens, mais au détriment de la qualité du routage, pouvant conduire à des acheminements dans des directions non optimales.

Exemple : Les messages OGM du nœud A se propagent vers le nœud B. La liaison étant asymétrique, B reçoit tous les paquets de A, contrairement à A qui ne reçoit quasiment rien de B. À mesure que tous les paquets de A parviennent à B, le nombre de paquets reçus par B augmente. B en déduit alors qu'il dispose d'une liaison parfaite vers A, ce qui est erroné. Dans le contexte de la communication avec une central de contrôle, si un message urgent devait être transmis ce n'est pas le genre de problème à négliger.



BATMAN IV résout ce problème en divisant la qualité d'une liaison en deux composantes distinctes : la qualité de réception et la qualité d'émission.

La qualité de réception exprime la probabilité de réussite de la transmission d'un paquet vers le nœud cible.

La qualité d'émission décrit la probabilité de réussite de la transmission vers un nœud voisin.

De ce fait la méthode d'inondation de paquets nous informe sur la qualité de la liaison de réception tandis que la qualité d'émission elle se mesure grâce à certains calculs.

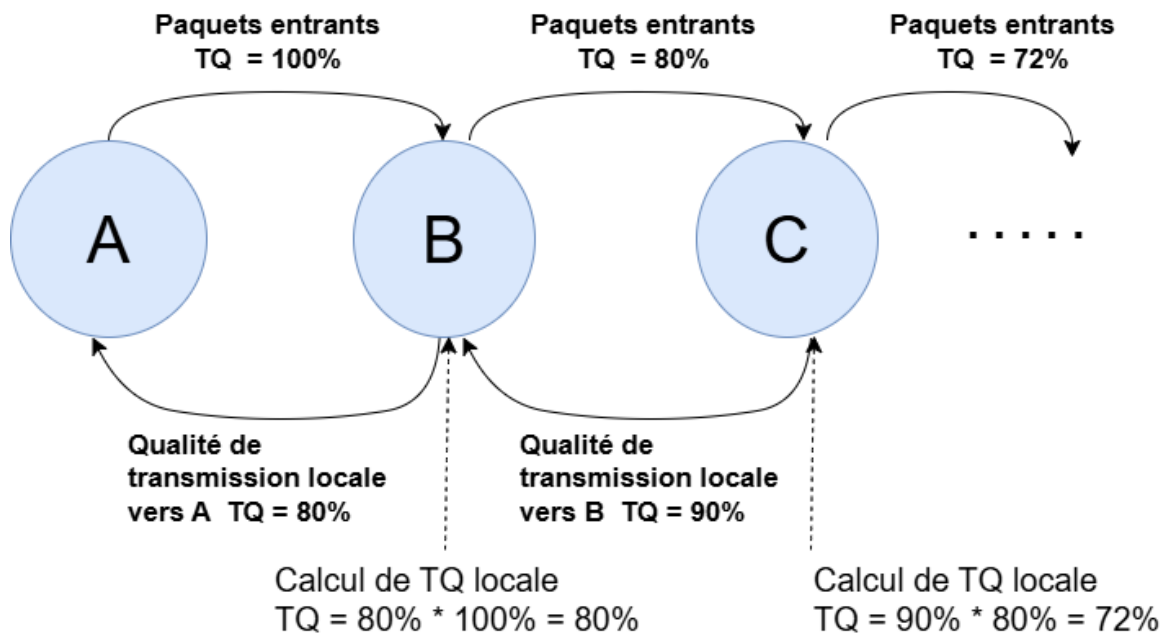
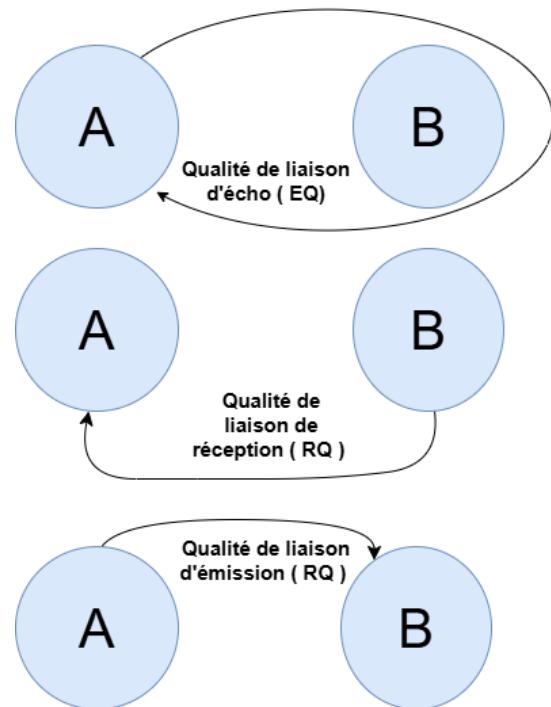
Dans le cas du noeud A :

On connaît la qualité de la liaison d'écho (EQ) en comptant les rediffusions de ses propres OGM par ses voisins.

On connaît la qualité de la liaison de réception (RQ) en comptant les paquets de ses voisins.

BATMAN peut calculer la qualité de la liaison d'émission (TQ) en divisant la qualité de la liaison d'écho par la qualité de la liaison de réception.

$$TQ = \frac{EQ}{RQ}$$



Afin de propager la qualité de liaison locale, Batman IV introduit dans les paquets envoyés un champ TQ de tel sorte qu'un noeud récepteur calcule sa propre qualité de liaison locale à partir de la valeur TQ reçue et retransmet le paquet. Ainsi, chaque noeud recevant un paquet connaît la qualité de transmission vers le noeud émetteur. $TQ = TQ_{entrant} * TQ_{local}$

L'objectif de ces mesures est de pénaliser les liaisons présentant une mauvaise qualité de

réception en évitant d'activer ou désactiver complètement une liaison avec des règles peu strictes ou trop strictes. Pour ce faire BATMAN IV utilise la fonction suivante.

$$f_{asym} = (100\% - (100\% - RQ)^3) \quad (1)$$

Cette valeur obtenue est multipliée par le TQ avant d'être rediffusée dans le cas où l'on applique cette pénalité.

Jusqu'à présent, BATMAN IV se concentre uniquement sur la qualité de la liaison pour évaluer les chemins, sans tenir compte du nombre de sauts, car il ignore la topologie au-delà de son horizon. Dans certaines configurations réseau, la qualité de la liaison des voisins est très similaire, contrairement au nombre de sauts.

Pour pallier ce problème, BATMAN IV introduit une pénalité de saut : à chaque passage d'un OGM à travers un nœud, la valeur TQ est diminuée d'une valeur fixe, indépendamment de la pénalité de liaison asymétrique, avant la retransmission du paquet.

3.5 Comparaison des différents protocoles

Maintenant que les différents protocoles de routage ont été présentés, il est temps de les comparer pour déterminer lequel correspond le mieux à notre situation. Pour rappel, nous cherchons un protocole de routage capable de mettre à jour l'état du réseau de façon autonome, et trouver le meilleur chemin pour faire circuler l'information, le tout dans un environnement souterrain où les communications peuvent être difficiles et imprévisibles. C'est sur la base de ces critères que nous allons maintenant les mettre en perspective.

Les différentes métriques avec lesquelles nous comparerons ces protocoles de routage seront les suivantes :

le coût de routage : Il mesure l'efficacité du protocole, si celui-ci émet beaucoup d'overhead et se définit comme le rapport entre le nombre de paquets de routage envoyés et le nombre de paquets de données reçus par les destinations. Les robots disposant d'une autonomie limitée,

un protocole générant trop d'overhead consommerait inutilement leurs ressources.

L'overhead d'un protocole est l'ensemble des données non applicatives (en-têtes, métadonnées ajoutées pour assurer le transport fiable, l'adressage et le contrôle de la communication.

le taux de paquets délivrés (mesure fiabilité de la transmission) : rapport entre le nombre de paquets de données reçus par les destinations et le nombre de données émis par des sources. Dans notre cas : une information perdue, qu'il s'agisse d'un relevé de radioactivité ou d'une alerte, pourrait avoir des conséquences graves.

le délai de bout en bout : c'est le temps écoulé entre l'envoi d'un paquet par un émetteur et sa réception par le destinataire . Si un robot détecte un seuil de radioactivité dangereux, l'information doit parvenir au centre de contrôle avec le moins de délai.

la gigue : c'est la différence du délai de deux paquets successivement reçus appartenant au même flux de données . Une gigue élevée rendrait les données de supervision difficiles à exploiter en temps réel.

La concentration de l'activité : permet d'évaluer la répartition de l'activité des nœuds mobiles dans le réseau Ad hoc. Ainsi, si la concentration de l'activité tend vers zéro, cela veut dire que la totalité des nœuds mobile travail équitablement. Elle se définit par la lettre A :

N : nombre total de noeuds du réseau ad hoc

$$A = \frac{\sum_{j=1}^N |n_j - \frac{1}{N} \sum_{i=1}^N n_i|}{\sum_{i=1}^N n_i}$$

n_i : le nombre de paquets retransmis ou envoyés par le noeud mobile M_i

Une répartition trop inégale signifierait que certains robots sont sollicités bien plus que d'autres, accélérant leur décharge et fragilisant l'ensemble de la flotte.

Les comparaisons qui suivront sont issues d'un document de recherche de l'INRIA (Laboratoire de recherche) disponibles dans la bibliographie pour plus de renseignement sur les

paramètres de tests utilisés. Ces résultats ne constituent pas une preuve formelle mais nous permettent néanmoins de dégager des tendances significatives que nous mettrons en perspective avec les contraintes spécifiques de notre environnement cible.

Les tests ont été effectués avec à **Nes2** qui est un **simulateur de réseau**.

Les paramètres principaux de ces tests sont les suivants :

Le trafic entre les nœuds est produit en utilisant un générateur de trafic qui crée aléatoirement des connexions qui commencent à des instants distribués uniformément entre 0 et 100 secondes.

Paramètre	Valeur
Type du trafic	CBR/UDP
Nombre de connexions	10, 20, 30, 40 et 50 connexions
Taux de transmission	8 paquets/seconde
Taille des paquets	512 octets

Random Waypoint : les nœuds choisissent une destination aléatoire (dans la zone de simulation), s'y rendent à une certaine vitesse (varie uniformément entre 0 et 20m/s), attendent un temps aléatoire, puis recommencent.

Paramètre	Valeur
Temps de simulation	100 secondes
Aire du réseau Ad hoc	1000m x 1000m
Nombre de nœuds	20, 40, 60, 80 et 100 nœuds
Temps de pause	0, 20, 40, 60, 80 et 100 secondes
Vitesse maximale des nœuds	20 m/s
Modèle de mobilité	Random Waypoint

Temps de Pause : c'est le temps d'attente entre chaque déplacement, le réduire augmente la mobilité.

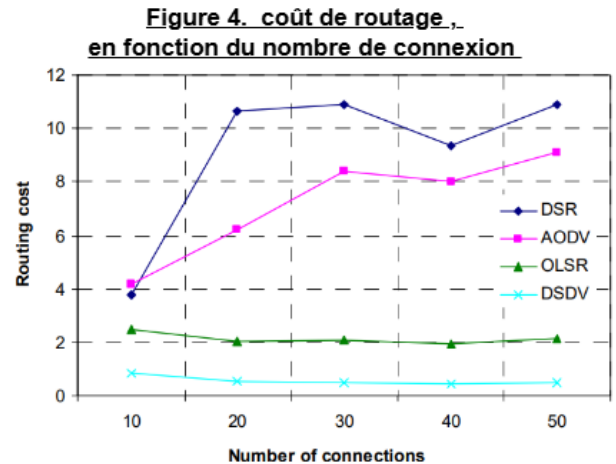
Il est important de noter que ces interprétations ont été effectuées par un étudiant en classe préparatoire et ne se veulent en aucun cas parfaitement rigoureuses. D'autres éléments pourraient être pris en compte dans l'interprétation de ces résultats.

Interprétation de la figure N°4 :

Le graphique représente le coût de routage en fonction du nombre de connexions, le nombre de nœuds étant fixe à 50.

Une connexion représente une session de communication active entre deux nœuds, et non un nœud supplémentaire dans le réseau.

AODV voit son coût augmenter entre 10 et 30 connexions en raison de l'overhead lié aux découvertes de routes initiales par inondation.



La stabilisation après 30 connexions s'explique par la réutilisation des routes en cache : une route découverte reste valide tant que le lien n'est pas rompu, limitant ainsi les redécouvertes. Les ruptures de liens dues à la mobilité maintiennent un overhead résiduel constant, ce qui explique que la courbe ne redescende pas.

OLSR maintient un coût quasi constant car ses messages de contrôle Hello et TC sont émis périodiquement selon la topologie, indépendamment du trafic. La mobilité entraîne des réélections de MPR, mais celles-ci restent locales et peu coûteuses grâce à la capacité d'un MPR à couvrir plusieurs voisins à deux sauts simultanément.

Dans notre contexte, où plusieurs robots communiquent simultanément pour superviser le site, ce comportement prévisible et stable est un avantage : quel que soit le nombre d'échanges en cours, le protocole ne génère pas de surcharge supplémentaire.

AODV en revanche, dont le coût augmente avec le nombre de connexions serait davantage pénalisé dans un scénario où de nombreux robots doivent échanger des données en parallèle, comme lors d'une supervision simultanée de plusieurs zones.

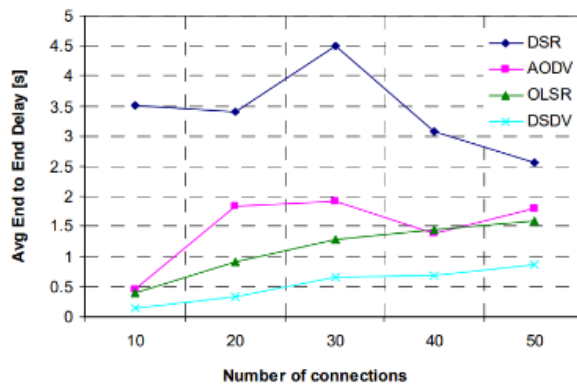


Figure 5. Délai , en fonction du nombre de connexion

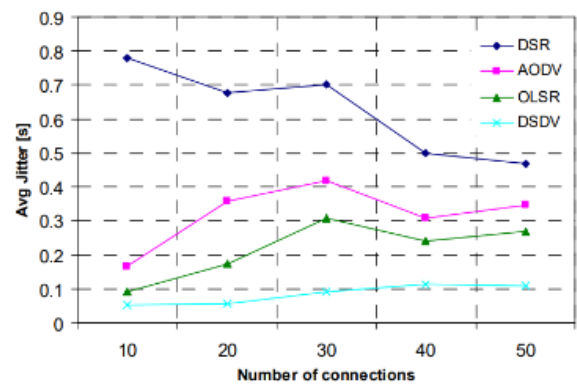


Figure 6. Gigue , en fonction du nombre de connexion

Interprétation de la Figure N°5 et N°6 :

Les figures 2 et 3 nous montrent qu'en termes de délai et de gigue, OLSR est plus performant qu'AODV car, en tant que protocole proactif, les routes vers les destinations sont connues en permanence, sans délai de découverte.

AODV en revanche, lorsque la route n'est pas en cache, doit déclencher une inondation avant d'acheminer le paquet, introduisant un délai supplémentaire. Ce délai atteint un pic vers 30 connexions, moment où la charge dépasse la capacité du cache à absorber toutes les destinations actives. La redescende après 30 connexions pourrait indiquer une saturation du réseau entraînant des abandons de paquets plutôt que des mises en attente, ce qui ferait artificiellement baisser le délai mesuré.

Dans notre contexte, Un protocole comme AODV, qui doit redécouvrir une route avant d'acheminer un message, pourrait introduire un retard inacceptable dans certaine situation d'urgence.

Interprétation de la figure N°7 :

La concentration d'activité est plus élevée pour OLSR car seuls les nœuds élus MPR sont responsables du relayage des messages de contrôle à travers le réseau. Cela entraîne une répartition inégale de la charge entre les nœuds. À l'inverse, AODV distribue cette charge plus uniformément car lors du processus d'inondation, tous les nœuds participent à la retransmission des requêtes de découverte de route.

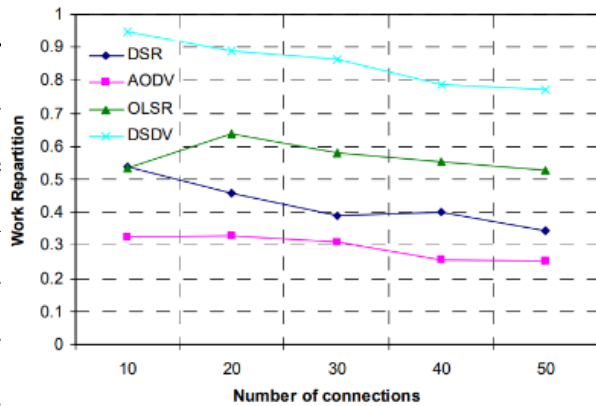


Figure 7. concentration de l'activité , en fonction du nombre de connexion

Interprétation de la figure N°8 :

les deux protocoles obtiennent des résultats similaires, l'augmentation du trafic devenant le facteur limitant pour les deux protocoles.

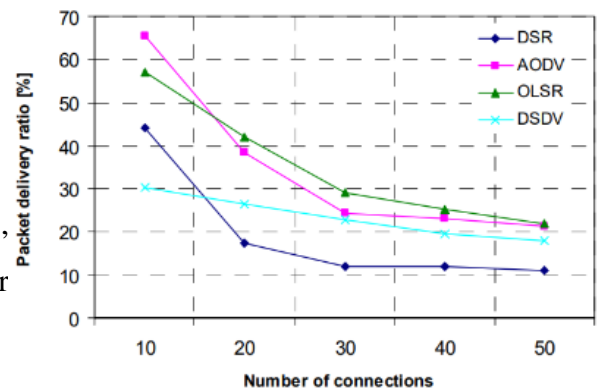


Figure 8. Taux de réception des paquets , en fonction du nombre de connexion

Synthèse

Au terme de cette comparaison, OLSR s'impose face à AODV sur les critères les plus critiques pour notre projet : il offre un délai et une gigue plus faibles, un coût de routage stable, et ne nécessite pas de redécouverte de route à chaque rupture de lien. Son seul désavantage, une concentration de l'activité sur les nœuds MPR, reste secondaire face aux risques qu'introduirait AODV en environnement souterrain. AODV est donc écarté. La question devient : OLSR est-il pour autant le protocole idéal, ou peut-on faire mieux ?

Comparaison OLSR / Batman

Problème inhérent d'OLSR

Dans le protocole OLSR, chaque nœud maintient une table de routage complète en diffusant régulièrement des messages de topologie (TC packets) à travers tout le réseau. Le problème est qu'il existe un décalage temporel entre la réalité du réseau et ce qui est stocké dans les tables de routage, car les messages de topologie mettent du temps à se propager. Ce décalage peut provoquer des boucles de routage, c'est à dire, un paquet tourne en rond entre plusieurs nœuds sans jamais atteindre sa destination.

BATMAN résout ce problème en supprimant complètement la diffusion de messages de topologie globale. Du fait que celui-ci se contente de connaître uniquement son meilleur voisin direct pour chaque destination, il n'a pas besoin de connaître la topologie globale du réseau. Cela élimine les boucles de routage.

En revanche, BATMAN introduit un autre inconvénient : une auto-interférence due au trafic de données, qui peut provoquer des oscillations de débit.

Présentation de l'expérience

Expérience issue d'un document scientifique (voir bibliographie)

L'expérience se déroule dans la cage d'escalier d'un bâtiment universitaire de 5 étages. Cinq ordinateurs portables sont déployés, un par étage, et communiquent entre eux via des connexions sans fil. Le nœud source se trouve au 5e étage et cherche à communiquer avec les nœuds des étages inférieurs, ce qui implique de traverser un nombre croissant de sauts : 1 saut pour le 4e étage, 2 pour le 3e, 3 pour le 2e, et 4 pour le 1er.

Deux scénarios ont été testés :

Scénario statique : les cinq nœuds restent fixes à leur étage pendant toute la durée de l'expérience. C'est le cas de référence, qui permet d'observer l'effet du nombre de sauts sans la variable de la mobilité.

Scénario mobile : seul le nœud du 1er étage reste fixe. Les quatre autres nœuds se déplacent d'un étage vers le bas, se remplaçant les uns les autres. Les nœuds s'arrêtent quelques secondes aux coins avant de reprendre leur déplacement. À la fin du mouvement, les nœuds se retrouvent plus proches les uns des autres au bas du bâtiment. Ce scénario simule une situation réelle où les équipements sont en mouvement.

Les données exploitées par la suite sont issues d'une revue scientifique de 2013 et ne font en aucun cas office de preuve formelle mais nous donne une idée sur les performances sachant que le protocole batman a beaucoup évolué Les détails conernants le matériel utilisé et les paramètres sont disponible en annexe.

TABLE 1 – Débit moyen (kb/s).

Flux	Étage	Scénario statique (STAS)		Scénario mobile (SHIS)	
		OLSR	BATMAN	OLSR	BATMAN
#1 → #2	4 ^e	409.6000	409.6000	409.5904	409.2450
#1 → #3	3 ^e	409.5304	407.2257	391.7681	345.1153
#1 → #4	2 ^e	232.4234	184.5043	166.2921	132.5324
#1 → #5	1 ^{er}	112.7911	76.7847	63.4341	146.5412

En scénario statique, pour 1 ou 2 sauts, les deux protocoles atteignent quasiment le débit maximal théorique. Les choses se dégradent significativement à partir de 3 sauts : le débit chute d'environ 50% pour les deux protocoles. OLSR reste globalement au-dessus de BATMAN, dont les oscillations sont plus marquées dès 2 sauts. En scénario mobile, la dégradation est encore plus forte (environ 60% de perte), mais on observe un phénomène intéressant : BATMAN remonte légèrement pour le flux le plus long (4 sauts), car à la fin du déplacement les nœuds sont physiquement plus proches et les liens de meilleure qualité.

TABLE 2 – Délai moyen (s).

Flux	Étage	Scénario statique (STAS)		Scénario mobile (SHIS)	
		OLSR	BATMAN	OLSR	BATMAN
#1 → #2	4 ^e	0.0045	0.0198	0.0209	0.0241
#1 → #3	3 ^e	0.0210	0.0885	0.4754	0.7207
#1 → #4	2 ^e	1.3185	2.6800	2.6742	3.6724
#1 → #5	1 ^{er}	1.6403	4.0517	3.5424	4.4339

En scénario statique, le délai augmente régulièrement avec le nombre de sauts pour les deux

protocoles. Pour 1 ou 2 sauts, les délais restent très faibles et quasi négligeables pour les deux protocoles. La dégradation devient significative à partir de 3 sauts : OLSR affiche un délai moyen de 1.32 s contre 2.68 s pour BATMAN. Pour 4 sauts, l'écart se creuse encore davantage, avec 1.64 s pour OLSR contre 4.05 s pour BATMAN.

En scénario mobile, les délais augmentent pour les deux protocoles dès 2 sauts, en raison des changements de routes fréquents provoqués par le déplacement des nœuds. Pour 3 ou 4 sauts, les délais moyens dépassent 2.5 s pour OLSR et 3.5 s pour BATMAN. OLSR reste donc systématiquement plus performant que BATMAN sur cette métrique, quel que soit le scénario considéré.

TABLE 3 – Perte de paquets moyenne (pps).

Flux	Étage	Scénario statique (STAS)		Scénario mobile (SHIS)	
		OLSR	BATMAN	OLSR	BATMAN
#1 → #2	4 ^e	0.0063	0.0000	0.0133	0.0867
#1 → #3	3 ^e	0.0170	0.4703	3.5033	10.7967
#1 → #4	2 ^e	18.5473	17.5783	29.8418	23.0733
#1 → #5	1 ^{er}	35.0030	38.7510	25.5562	18.8267

En scénario statique, pour 1 ou 2 sauts, les pertes de paquets sont quasi nulles pour les deux protocoles, ne dépassant pas 0.47 pps. La situation se dégrade significativement à partir de 3 sauts, avec des valeurs moyennes comprises entre 17 et 39 pps pour les flux à destination des nœuds #4 et #5.

En scénario mobile, les pertes augmentent globalement par rapport au scénario statique, en raison des changements de routes fréquents provoqués par le déplacement des nœuds.

Cependant, on observe un phénomène notable pour le flux #1 → #5 : BATMAN affiche une perte de paquets inférieure à celle d'OLSR (18.83 pps contre 25.56 pps). Cela s'explique par le mécanisme de tampon (buffer) de BATMAN, qui conserve temporairement les paquets lorsque les routes sont instables plutôt que de les abandonner immédiatement. Toutefois, ce mécanisme se fait au détriment du délai, qui continue d'augmenter.

Dans l'ensemble, OLSR présente de meilleures performances que BATMAN sur cette métrique, à l'exception du flux #1 → #5 en scénario mobile où le buffering de BATMAN lui confère un avantage ponctuel.

Synthèse

OLSR performe mieux que BATMAN sur presque toutes les métriques mesurées. Pourtant, sa vulnérabilité aux boucles de routage, inhérente à sa connaissance globale de la topologie,

constitue un risque structurel que les chiffres seuls ne reflètent pas. BATMAN, en ne raisonnant qu'à l'échelle de son voisin direct, élimine ce risque par conception. Pour des flux légers comme les nôtres, ses performances légèrement inférieures sont acceptables ; une boucle de routage, elle, ne l'est pas. C'est ce qui motive le choix de BATMAN IV pour la campagne de tests.

4 Campagne de tests

4.1 Environnement

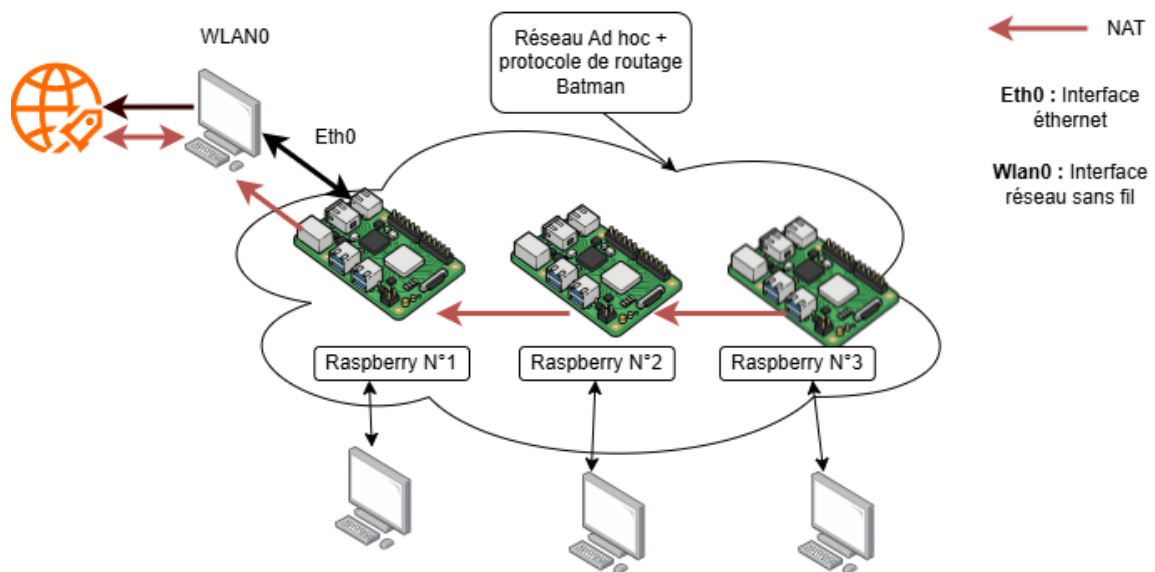


Figure N°8 : Environnement d'expérimentation

Les précédentes comparaisons sont issues de travaux de recherche souvent assez anciens. Depuis, de nouvelles versions de ces protocoles sont sorties, corrigeant des problèmes inhérents aux versions précédentes comme on a pu le voir pour le protocole BATMAN.

L'objectif de cette partie sera donc la mise en place d'un environnement de test en réseau ad hoc avec le protocole de routage BATMAN IV. L'objectif étant de tenter de reproduire un environnement de test qui tende au mieux à représenter différentes contraintes que l'on pourrait rencontrer dans l'exploration de zones d'enfouissement. Pour ce faire, on utilise une architecture simple composée de Raspberry Pi 3 sous ubuntu 24.04

Pour ce faire, j'ai créé un NAT entre le Raspberry N°1 et un 1er PC ayant accès à internet permettant à celui-ci d'accéder à Internet en Ethernet via le PC. J'ai ensuite relié les deux autres Raspberry (N°2 et N°3) sous forme de chaîne. On a associé un Pc à chaque raspberry afin de pouvoir les manipuler lors des tests

Exemple : le Raspberry N°2 a besoin d'Internet pour télécharger un logiciel. Il passe alors par le Raspberry N°1, grâce au réseau ad hoc, qui est lui-même relié en Ethernet au PC disposant d'une connexion Internet.

Remarque : il n'était pas obligatoire de relier les Raspberry en chaîne. Le Raspberry N°3 aurait très bien pu passer directement par le Raspberry N°1 ; tout dépend de la configuration choisie.

Enfin, j'ai configuré le réseau ad hoc ainsi que le protocole de routage BATMAN IV.

La démarche concernant la mise en place du NAT, la configuration complète du réseau BAT-MAN ainsi que les fichiers d'automatisation seront disponibles sur mon GitHub (voir annexe).

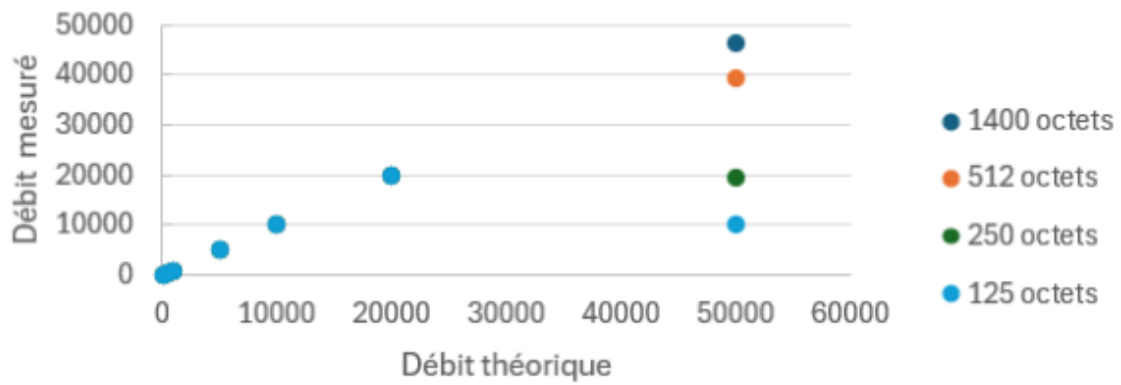
4.2 Protocoles expérimentaux

Un premier Raspberry Pi est placé au premier étage d'une résidence de neuf étages et sert de nœud source fixe. À chaque étage, un second Raspberry Pi est déplacé et un script est exécuté afin de mesurer les métriques suivantes : le débit, la taille des données échangées, la variation du délai ainsi que le taux de paquets perdus. Pour chaque étage, les mesures sont effectuées en faisant varier deux paramètres indépendamment. La taille des paquets prend successivement les valeurs suivantes : 125, 250, 512 et 1400 octets. Le débit cible est quant à lui testé selon les valeurs suivantes : 100 kbps, 500 kbps, 1 Mbps, 5 Mbps, 10 Mbps, 20 Mbps et 50 Mbps. Les résultats des expériences sont consultables via le github présent dans la bibliographie.

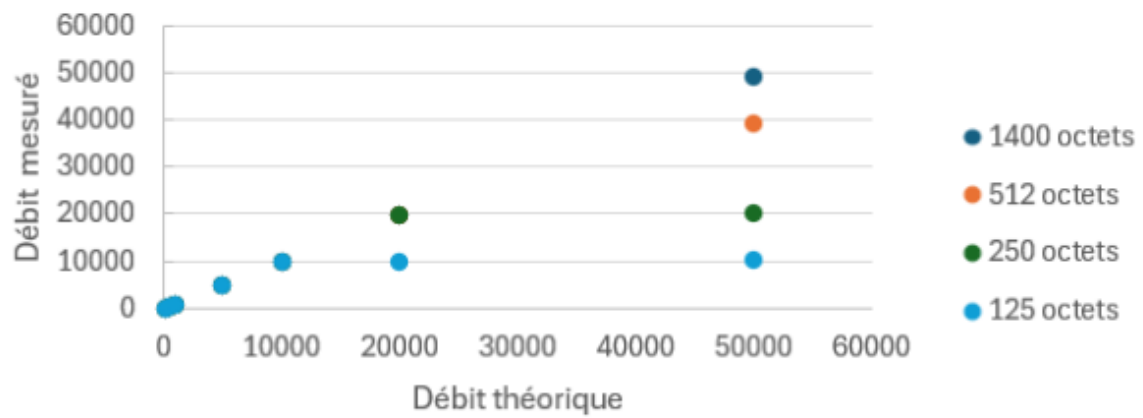
5 Présentation des résultats

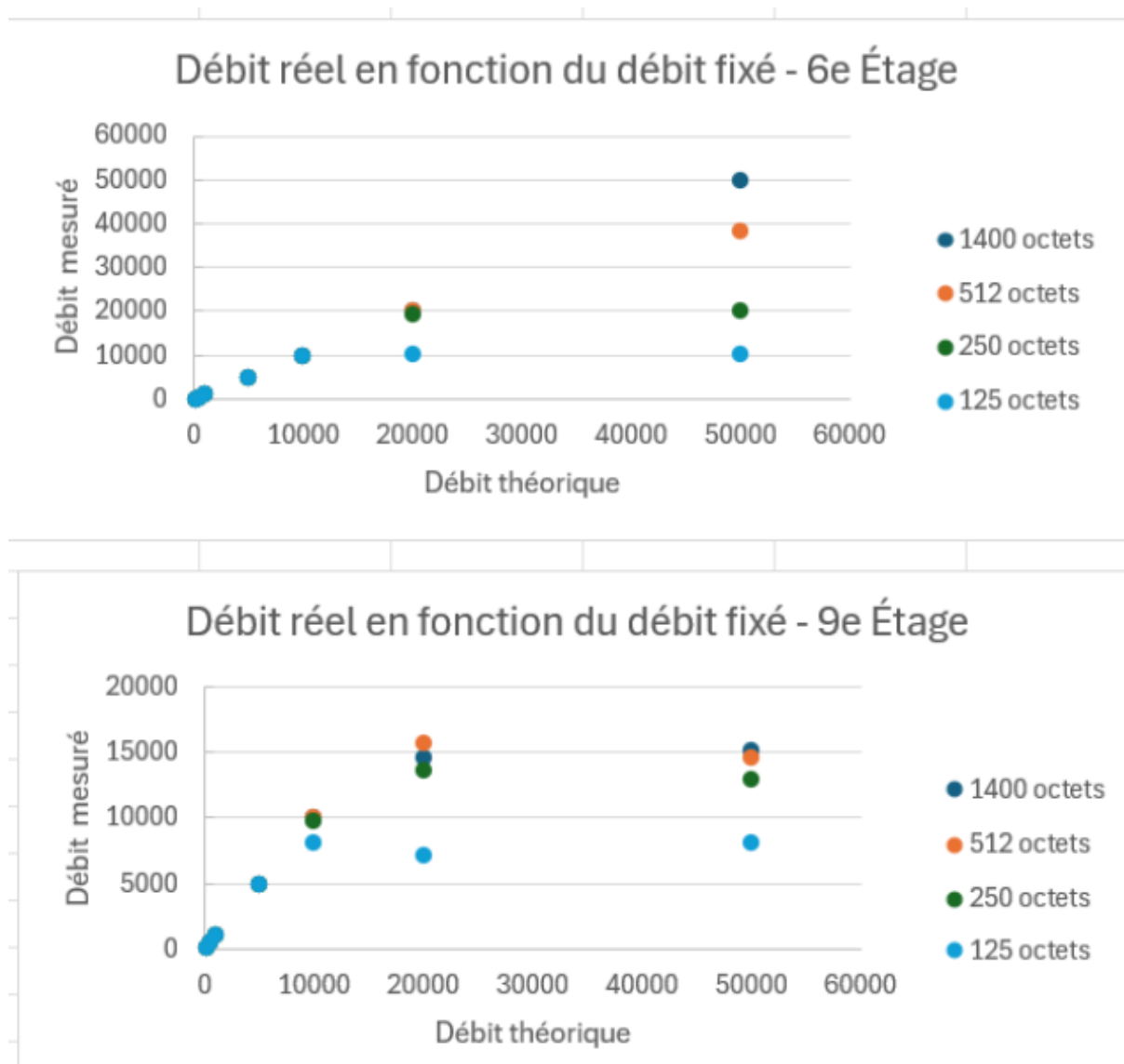
Figure N°9 : Impact de la distance et de la taille des paquets sur le débit réel mesuré

Débit réel en fonction du débit fixé - 2e Étage



Débit réel en fonction du débit fixé - 4e Étage





Jusqu'à 5 Mbps, pour des débits modérés, BATMAN IV assure une transmission parfaite quelle que soit la distance ou la taille des paquets. Dans le cadre de notre projet, cela est important car la transmission d'alertes ou de relevés de capteurs, qui constituent des flux légers, serait garantie sans perte sur l'ensemble de la galerie.

À partir de 10 Mbps, les comportements commencent à diverger selon la distance et la taille des paquets. Aux étages 2, 4 et 6, seuls les petits paquets sont affectés à des débits élevés, en raison de l'overhead de routage proportionnellement plus important par octet transmis.

Le 9^e étage constitue en revanche un point de rupture : la dégradation touche toutes les tailles de paquets dès 10 Mbps, avec des pertes atteignant 84% à 50 Mbps pour les plus petits paquets. Dans une galerie souterraine, où les obstacles aggravent l'atténuation du signal, cette

limite serait probablement atteinte bien plus tôt, soulignant la nécessité de prévoir suffisamment de robots intermédiaires pour relayer les communications.

Impacte de l'ajout d'un noeud relais sur le délai

Cette nécessité se retrouve également dans le taux de paquets perdus. En effet, un test mené entre un Raspberry Pi placé au 9^e étage et un autre au 1^{er} étage a permis d'obtenir un taux de paquets perdus de 36%. L'ajout, dans les mêmes conditions, d'un nœud relais a permis de ramener ce taux à 6,6%, confirmant l'importance d'une infrastructure de relais adaptée.

Cout de routage

Sur l'ensemble des tests effectués, j'ai observé que l'overhead représentait en moyenne entre 35-40 %, pour 100 octets envoyés, 40 sont consacrés aux messages de contrôle du protocole ce qui reste correcte pour des alertes et flux à faible débit. La question serait de voir à quel point ceci a un impact sur l'autonomie. Paramètre que je ne peux pas déterminer.

D'autres observations méritent d'être mentionnées :

La première concerne la latence de mise à jour de la topologie : bien que BATMAN soit conçu pour corriger les lenteurs de convergence d'OLSR, j'ai constaté de nombreuses reprises que les informations de voisinage de mes Raspberry Pi étaient erronées. Un nœud pouvait rester détecté comme voisin pendant un certain temps après être sorti de portée, ce qui introduit un risque de routes fantômes dans le réseau. La seconde observation porte sur le comportement du protocole en limite de portée. Lorsque la qualité de liaison entre deux nœuds devient insuffisante, BATMAN ignore complètement ce voisin plutôt que de dégrader progressivement la communication. Pour rétablir la liaison, les deux nœuds doivent se rapprocher significativement, ce qui dans le cadre d'une flotte de robots autonomes pourrait contraindre leurs déplacements. Enfin, l'environnement de test s'est révélé déterminant. Dans les espaces ouverts du laboratoire LORIA, la portée du réseau s'est étendue à l'ensemble du bâtiment, tandis que les tests effectués au sein d'une résidence, avec de nombreuses cloisons et infrastructures, ont drastiquement réduit cette portée. Ce contraste confirme que les performances de BATMAN IV en galerie souterraine seraient significativement différentes de celles mesurées en environnement ouvert.

6 Conclusion

Ce stage a permis d'étudier trois protocoles de routage : AODV, OLSR et BATMAN puis de les évaluer expérimentalement sur une architecture de Raspberry Pi en environnement cloisonné. En réponse à la problématique, Le protocole de routage BATMAN (version IV et plus) s'impose comme le choix le plus pertinent. Il évite les boucles de routage d'OLSR et les délais de découverte d'AODV, tout en garantissant une transmission fiable pour les flux légers caractéristiques du projet, alertes et relevés de capteurs, Son adoption reste néanmoins conditionnée à un maillage suffisant de robots-relais : les tests ont montré qu'au-delà d'une certaine distance, la dégradation des liens devient significative, et que l'ajout d'un seul nœud intermédiaire suffit à diviser le taux de perte par cinq. Enfin, les performances mesurées en bâtiment ouvert constituent un cas favorable ; en galerie souterraine, l'atténuation du signal sera plus sévère et devra être compensée par une densité de déploiement adaptée.

7 Bibliographie

Lien Github du projet :

<https://github.com/Amjad-Fanid>

Documents concernant AODV :

- <https://theses.hal.science/tel-01266383v1>
- <https://datatracker.ietf.org/doc/rfc3561/>

Documents concernant OLSR :

- <https://inria.hal.science/inria-00471712/>
- <https://datatracker.ietf.org/doc/html/rfc3626>

Documents concernant BATMAN :

- https://www.open-mesh.org/doc/batman-adv/BATMAN_I_V.html

Documents concernant la mise en place du banc de tests :

- <https://wiki.reseaulibre.ca/documentation/ad-hoc/>
- <https://www.worteks.com/blog/2025-voyage-borne-wifi-mesh/>
- <https://wiki.reseaulibre.ca/documentation/batman/>
- <https://www.linkedin.com/pulse/batman-adv-mesh-setup-tutorial-raspberry-pi-20222025-robi-sen-vshrc>
- <https://www.open-mesh.org/projects/batman-adv/wiki/Using-batctl>

Documents concernant les expériences analysées :

- <https://inria.hal.science/inria-00468484/document>
- <https://www.sciencedirect.com/science/article/pii/S089812211100589X>